

Team

1: Foundation Layer

DMS Team 1

July 2026

data.c data.h display.c display.h — Build Guide

Team 1 — Foundation Layer

You build the **shared foundation** everyone else depends on: **data.c**, **data.h**, **display.c**, and **display.h**. Without your files, Team 2 and Team 3 cannot compile or run menus.

Your deliverable: 4 files that hold ALL product/dealer/retailer data and ALL screen printing (menus, tables, password).

Team	Files	Role
Team 1 (YOU)	data.c, data.h, display.c, display.h	Database + UI helpers
Team 2	main.c, company.c, company.h	Program start + Company panel
Team 3	dealer.c, dealer.h, retailer.c, retailer.h	Dealer + Retailer panels + ordering

Three-team split for DMS

1. What You Are Building

1.1 data.h — The data contract

Defines **limits** (MAX_PRODUCTS, MAX_DEALERS, MAX_RETAILERS) and declares every shared array with **extern**. Other teams include this file to read stock and names.

1.2 data.c — The real database

Defines the actual arrays in memory and **init_data()** to load sample Rice/Oil/Sugar, two dealers, two retailers, and all starting stock. Also provides **show_company_products()** and **show_dealer_products()** for ordering screens.

1.3 display.h + display.c — Shared UI

Every menu box, table row, success/error message, password check, and order summary. Team 2 and 3 never draw tables themselves — they call your functions.

2. Build Order (Do This Sequence)

1. Create **data.h** — constants and extern declarations only.
2. Create **data.c** — arrays + `init_data()` with sample data.
3. Create **display.h** — list every print function.
4. Create **display.c** — implement prints + `verify_password()`.
5. Test with a tiny **test_main.c** (optional) or wait for Team 2's `main.c`.

Finish data.h before Team 2/3 start their .c files — they need your extern names and MAX_* values.

3. Step-by-Step: data.h

Start empty. Add one section at a time.

Step 3.1 — Limits

C

```
#define MAX_PRODUCTS 10
#define MAX_DEALERS 5
#define MAX_RETAILERS 5
```

Step 3.2 — Product arrays (extern)

C

```
extern int product_id[MAX_PRODUCTS];
extern char product_name[MAX_PRODUCTS][50];
extern int company_stock[MAX_PRODUCTS];
extern int product_total;
```

Step 3.3 — Dealer arrays

C

```
extern int dealer_id[MAX_DEALERS];
extern char dealer_name[MAX_DEALERS][50];
extern int dealer_stock[MAX_DEALERS][MAX_PRODUCTS];
extern int dealer_total;
```

Step 3.4 — Retailer arrays

C

```
extern int retailer_id[MAX_RETAILERS];
extern char retailer_name[MAX_RETAILERS][50];
extern int retailer_dealer[MAX_RETAILERS];
extern int retailer_stock[MAX_RETAILERS][MAX_PRODUCTS];
extern int retailer_total;
```

Step 3.5 — Function declarations

C

```
void init_data();  
void show_company_products();  
void show_dealer_products(int dealer_index);
```

4. Step-by-Step: data.c

Step 4.1 — Includes and array definitions

C

```
#include <stdio.h>
#include <string.h>
#include "data.h"
#include "display.h"

int product_id[MAX_PRODUCTS];
char product_name[MAX_PRODUCTS][50];
int company_stock[MAX_PRODUCTS];
int product_total;

int dealer_id[MAX_DEALERS];
char dealer_name[MAX_DEALERS][50];
int dealer_stock[MAX_DEALERS][MAX_PRODUCTS];
int dealer_total;

int retailer_id[MAX_RETAILERS];
char retailer_name[MAX_RETAILERS][50];
int retailer_dealer[MAX_RETAILERS];
int retailer_stock[MAX_RETAILERS][MAX_PRODUCTS];
int retailer_total;
```

Step 4.2 — init_data() products

C

```
void init_data() {
    product_total = 3;
    dealer_total = 2;
    retailer_total = 2;

    product_id[0] = 1;
    strcpy(product_name[0], "Rice");
    company_stock[0] = 100;

    product_id[1] = 2;
    strcpy(product_name[1], "Oil");
    company_stock[1] = 80;

    product_id[2] = 3;
    strcpy(product_name[2], "Sugar");
    company_stock[2] = 60;
```

Step 4.3 — init_data() dealers + stock grid

C

```
dealer_id[0] = 1;
strcpy(dealer_name[0], "Dhaka Dealer");
dealer_id[1] = 2;
strcpy(dealer_name[1], "Chittagong Dealer");

dealer_stock[0][0] = 20; dealer_stock[0][1] = 15; dealer_stock[0][2] = 10;
dealer_stock[1][0] = 25; dealer_stock[1][1] = 12; dealer_stock[1][2] = 18;
```

Step 4.4 — init_data() retailers

C

```
retailer_id[0] = 1;
strcpy(retailer_name[0], "Mirpur Shop");
retailer_dealer[0] = 1;

retailer_id[1] = 2;
strcpy(retailer_name[1], "Gulshan Shop");
retailer_dealer[1] = 2;

retailer_stock[0][0] = 5; retailer_stock[0][1] = 3; retailer_stock[0][2] = 2;
retailer_stock[1][0] = 4; retailer_stock[1][1] = 6; retailer_stock[1][2] = 1;
}
```

Step 4.5 — show_company_products()

Loop with for, call display helpers:

C

```
void show_company_products() {
    int i;
    print_subtitle("Company Products (Available to Order)");
    print_table_product_header();
    for (i = 0; i < product_total; i++) {
        print_table_product_row(product_id[i], product_name[i], company_stock[i]);
    }
    print_table_footer();
}
```

Step 4.6 — show_dealer_products(dealer_index)

C


```
void show_dealer_products(int dealer_index) {
    int i;
    print_subtitle("Dealer Products (Available to Order)");
    printf("Dealer: %s\n", dealer_name[dealer_index]);
    print_table_product_header();
    for (i = 0; i < product_total; i++) {
        print_table_product_row(product_id[i], product_name[i],
                                dealer_stock[dealer_index][i]);
    }
    print_table_footer();
}
```

5. Step-by-Step: display.h and display.c

display.h — declare all helpers

C

```
void print_line();
void print_menu_header(char title[]);
void print_menu_item(int number, char label[]);
void print_subtitle(char subtitle[]);
void print_table_product_header();
void print_table_product_row(int id, char name[], int stock);
void print_table_dealer_header();
void print_table_dealer_row(int id, char name[]);
void print_table_retailer_row(int id, char name[], int dealer_id);
void print_table_footer();
void print_success(char message[]);
void print_error(char message[]);
int verify_password();
void print_order_summary(char product[], int quantity, char from[], char to[]);
```

display.c — password (Team 2 & 3 need this)

C

```
#define LOGIN_PASSWORD "1122"

int verify_password() {
    char password[20];
    printf("Enter Password: ");
    scanf("%s", password);
    if (strcmp(password, LOGIN_PASSWORD) == 0) return 1;
    print_error("Invalid password.");
    return 0;
}
```

Table row — core pattern

C

```
void print_table_product_row(int id, char name[], int stock) {
    printf("| %-4d | %-20s | %-10d |\n", id, name, stock);
}
```

6. Test Your Part Alone

Before Team 2 delivers main.c, compile only your files with a minimal test:

C

```
/* test_team1.c – delete after integration */
#include <stdio.h>
#include "data.h"
#include "display.h"

int main() {
    init_data();
    print_menu_header("Team 1 Test");
    show_company_products();
    if (verify_password()) print_success("Password OK");
    return 0;
}
```

BASH

```
gcc test_team1.c data.c display.c -o test1
./test1
```

Expected: product table with Rice/Oil/Sugar, then password prompt. Type 1122 → SUCCESS.

7. Handoff to Team 2 and Team 3

You give them	They use it for
data.h	Access to all arrays and init_data()
display.h	Menus, tables, password, order summary
init_data()	Must be called once at program start
company_stock[], dealer_stock[], retailer_stock[]	Order logic changes these
show_company_products()	Dealer ordering screen
show_dealer_products(i)	Retailer ordering screen

What teammates need from you

Team 1 completion checklist

- data.h has all extern arrays and MAX_* defines.
- init_data() loads 3 products, 2 dealers, 2 retailers with correct stock.
- All display functions compile with no warnings.
- verify_password() works with 1122.
- Shared folder: commit data.c, data.h, display.c, display.h.